Kafka і подієвий стрімінг



Заняття №20

Node.js Pro

Лектор
Кирило Орлов

r_d

ПЛАН ЗАНЯТТЯ



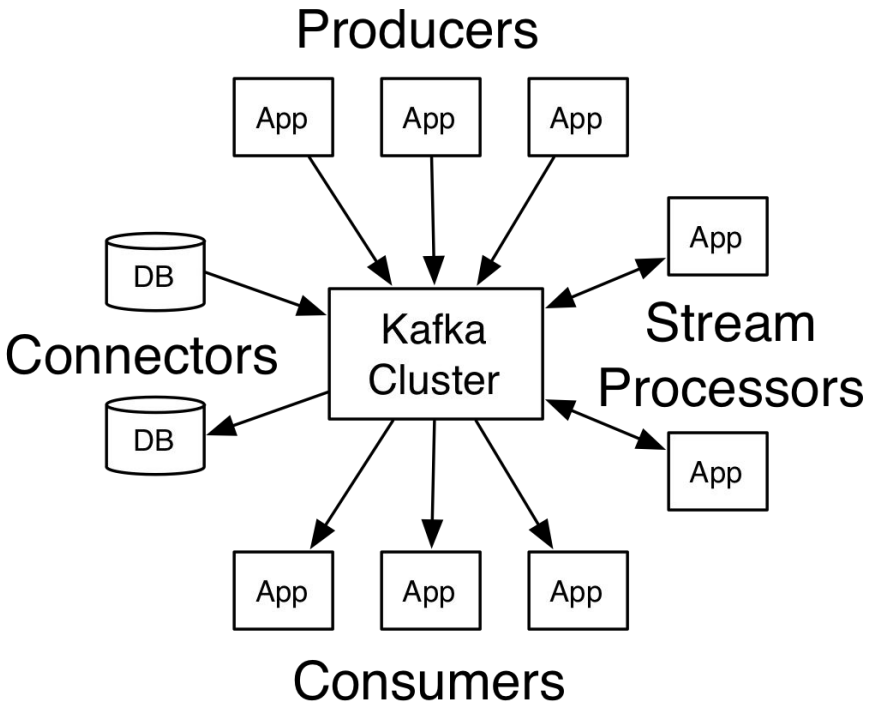
- Лог проти черги: чому commit нічого не видаляє
- Топіки, партиції, ключі: порядок і паралельність
- Група і rebalance: хто що читає і чому зупиняється
- Офсети: чому вибір завжди між дублем і втратою
- Retention і компакція: як журнал не росте вічно
- Exactly-once чесно + Kinesis, Pulsar, Redpanda

ЛОГ ПРОТИ ЧЕРГИ



ACK ВИДАЛЯЄ. COMMIT — НІ

KAFKA — НЕ ШВИДКА ЧЕРГА, А ЖУРНАЛ, ЩО ЗБЕРІГАЄТЬСЯ



ЧЕРГА І ЛОГ — РІЗНІ МОДЕЛІ

ШІСТЬ РІЗНИЦЬ, ІЗ ЯКИХ ВИПЛИВАЄ ВСЯ ЛЕКЦІЯ

Критерій	Kafka	RabbitMQ
Споживання	commit зсуває офсет, дані лишаються	ack видаляє повідомлення
Споживачів на подію	Скільки завгодно незалежних груп	Рівно один
Порядок	Лише в межах партиції	Глобально по черзі
Хто пам ʼятає прогрес	Споживач: офсет належить групі	Брокер
Перечитати вчорашнє	seek на офсет або таймстемп	Неможливо — дані вже зникли
DLQ	Немає з коробки — пишете самі	Вбудований DLX

ЖОДНОГО ЗНИКЛОГО ЗАПИСУ

ЖИВИЙ ПРОГІН: ОДИН ТОПІК, ДВІ ГРУПИ

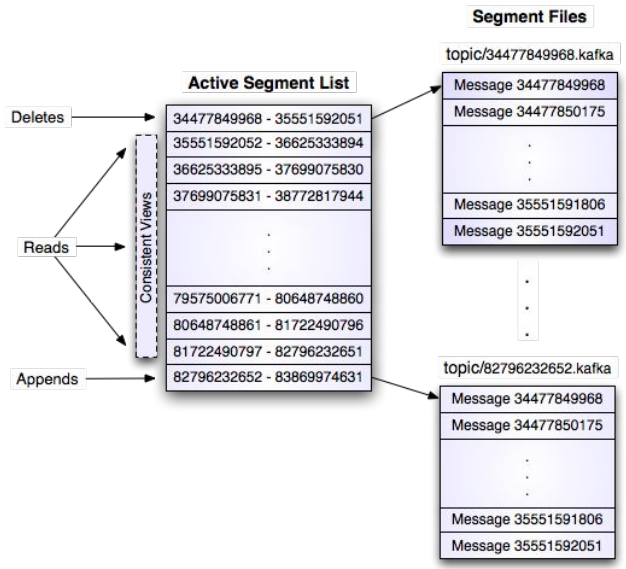
Група warehouse читає з початку → 5 подій

Та сама група читає ще раз → 0: закладка вже в кінці логу

Нова група analytics → знову 5: дані на місці

Нова група без fromBeginning → 0: це auto.offset.reset, а не перемотка

Було low=0 high=5 · стало low=0 high=5: commit пересунув ЧИСЛО, а не дані



ТОПІКИ, ПАРТИЦІЇ, КЛЮЧІ



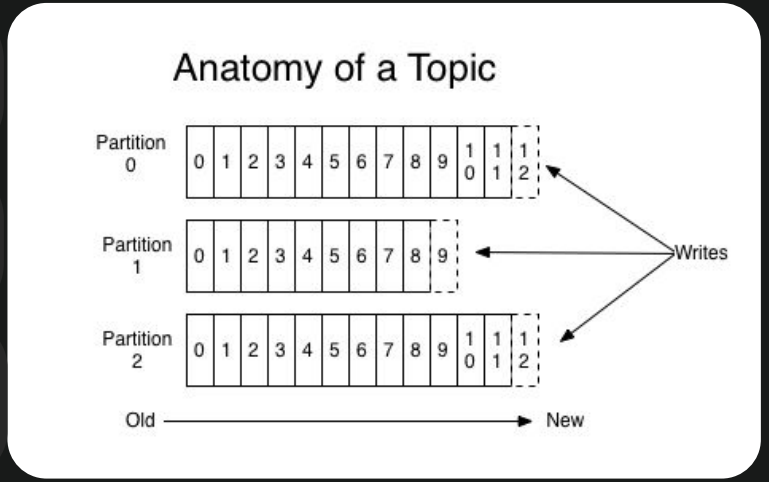
ТРИ ТВЕРДЖЕННЯ ПРО ПАРТИЦІЇ

ПАРТИЦІЯ — НАЙДРІБНІШЕ, ЩО ВІДДАЮТЬ СПОЖИВАЧЕВІ

Порядок гарантований лише в межах партиції — і ніде більше

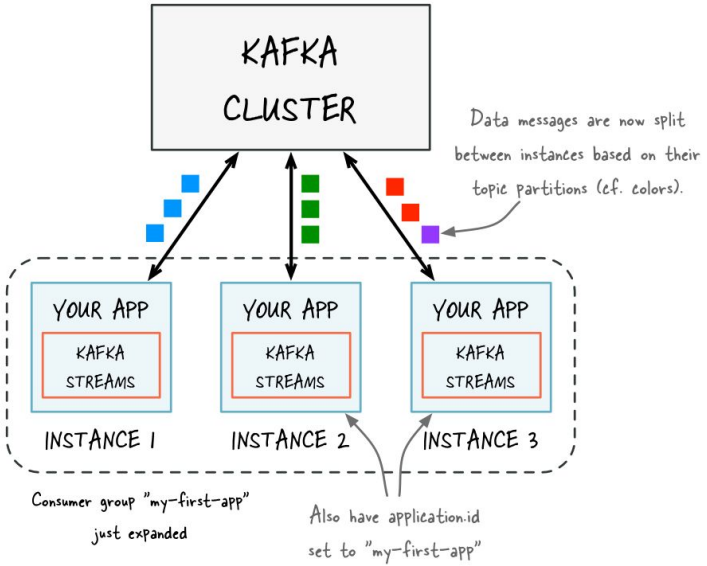
У межах групи партицію читає не більш ніж один консюмер

Отже активних консюмерів у групі не більше, ніж партицій



6 ПАРТИЦІЙ, 20 ПОДІВ → 14 БЕЗ РОБОТИ

НЕ «ПОВІЛЬНІШЕ» — ЦІ 14 БУКВАЛЬНО НІЧОГО НЕ РОБЛЯТЬ



КЛЮЧ → ХЕШ → ПАРТИЦІЯ

ПАРТИЦІЮ ОБЧИСЛЮЄ КЛІЄНТ, А НЕ БРОКЕР

Правило: "Однаковий ключ → та сама партиція → порядок для цього ключа збережено"

KafkaJS-шар рахує murmur2, сирий rdkafka — CRC-32: різні партиції для того самого ключа

Без ключа працює sticky partitioner, а не round-robin: «події поруч» не гарантовані



Гарячий ключ: 260 подій в одній партиції проти 5 у сусідній

Демо: у межах ОДНОГО npm-пакета 2 із 3 ключів пішли в різні партиції

Міні-вправа: Який ключ обрати для OrderPlaced, OrderStatusChanged, PaymentCaptured — і що станеться, якщо в той самий топік пише ще й Java-сервіс?

ГРАБЛІ ПАРТИЦІЙ

СКІЛЬКИ ПАРТИЦІЙ — ЦЕ МЕТОД, А НЕ ЧИСЛО

Додати партиції можна, прибрати — заборонено: брокер відповідає «2 would not be an increase»

Розширення ламає відображення ключів: 7 із 12 ключів переїхали в іншу партицію

Стеля кардинальності: ключ із трьох значень — 3 партиції з роботою, 3 порожні назавжди

Анти-патерн «у нас лаг — піднімемо ще подів»: спершу kafka-consumer-groups --describe

Метод: виміряти пропускну здатність ОДНІЄЇ партиції на своєму залізі, поділити цільову, додати запас. Число з голови — червоний прапор

ГРУПА, REBALANCE, ОФСЕТИ



ГРУПА І REBALANCE

ТРИ ПОКОЛІННЯ ПРОТОКОЛУ І ТРИ ТАЙМАУТИ

group.id

Не ім ʼя процесу, а відповідь на питання «хто ми як читач»

Eager (RangeAssignor)

Усі віддають усі партиції: поки триває — не споживає ніхто

Cooperative (KIP-429)

Забирають лише ті партиції, що реально переїжджають

KIP-848

Розподіл рахує брокер. GA з 4.0, але дефолт group.protocol у 4.3 — досі classic

Static membership

group.instance.id: под, що швидко повернувся, забирає ті самі партиції

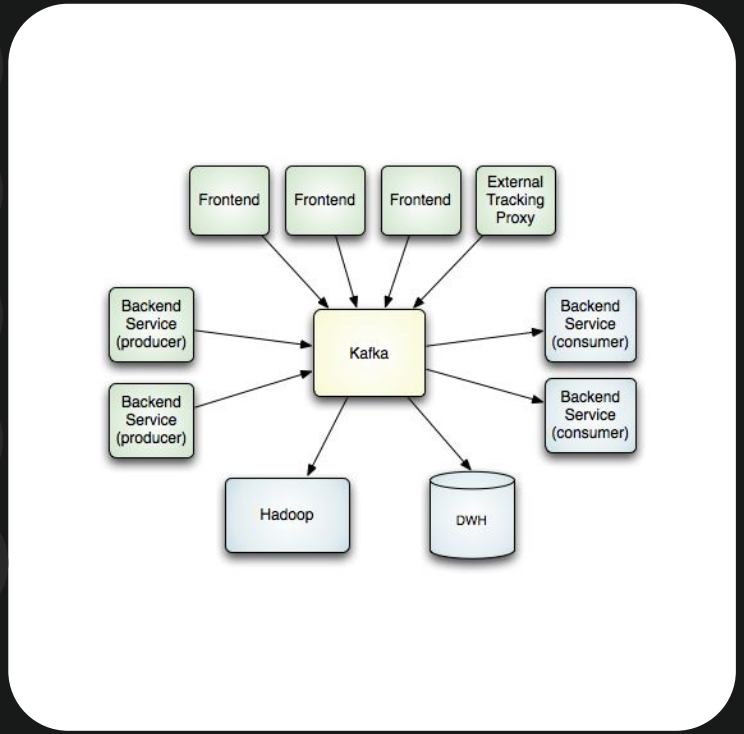
Три таймаути

heartbeat 3000 мс · session.timeout.ms 45000 · max.poll.interval.ms 5 хв

ТРИ ЧИСЛА, ЯКІ ПЛУТАЮТЬ

ОФСЕТ — ВЛАСТИВІСТЬ ПАРИ (ГРУПА, ПАРТИЦІЯ)

- offset запису: незмінний порядковий номер усередині партиції
- position: звідки консюмер прочитає наступного разу — живе в пам ʼяті процесу
- committed offset: звідки почне наступний власник партиції після rebalance
- Лежить у __consumer_offsets — звичайному компактованому топіку
- Наслідок — позначити повідомлення обробленим вибірково НЕ МОЖНА — закладка це одне число



ПОРЯДОК ДВОХ РЯДКІВ = СЕМАНТИКА

ЦЕ НЕ НАЛАШТУВАННЯ БРОКЕРА, А ТЕ, ЩО ВИ НАПИСАЛИ ПЕРШИМ

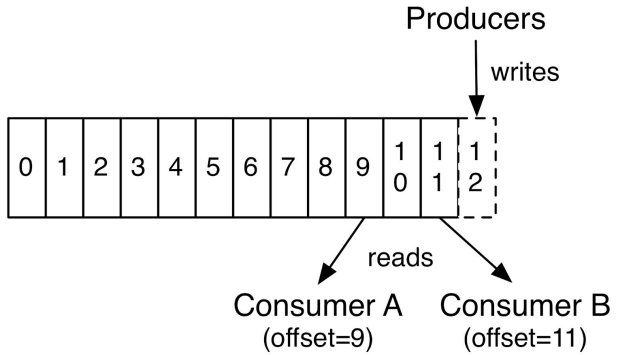
Типові режими:

at-most-once: спершу commit → падіння = втрата

at-least-once: спершу handle → падіння = дубль

Третього на рівні доставки немає

Rebalance — штатна подія: саме в ній народжуються дублі



ЩО ПОКАЗАВ SIGKILL

ТРИ СЦЕНАРІЇ, ОДНА РІЗНИЦЯ — ПОРЯДОК ДВОХ РЯДКІВ

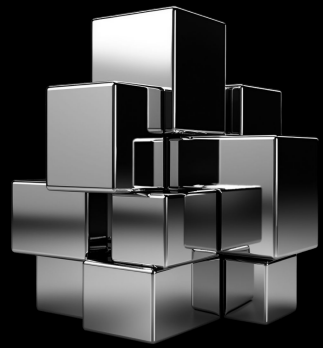
```
// at-most-once — спершу коміт, потім обробка
commitOffsets();   await handle(msg);

// → впав між рядками: committed 3, msg-3 ВТРАЧЕНО НАЗАВЖДИ

// at-least-once — спершу обробка, потім коміт
await handle(msg); commitOffsets();
// → впав між рядками: committed 4, ДУБЛЬ msg-5, втрат 0

// дефолт клієнта: enable.auto.commit = true, інтервал 5 с
// → SIGKILL у вікні автокоміту: 5 подій → 10 доставок

// це не налаштування брокера, а те,
// який із двох рядків ви написали першим
```



RETENTION ПРОТИ КОМПАКЦІЇ



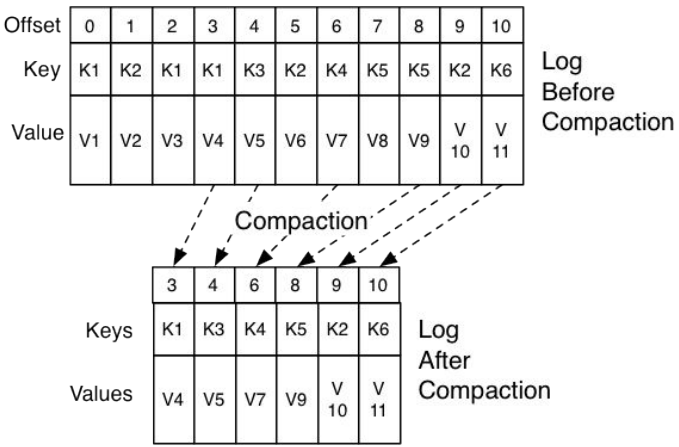
ДВА РІЗНІ ПРИБИРАЛЬНИКИ

RETENTION ЗНІМАЄ ЗА ВІКОМ, КОМПАКЦІЯ — ЗА КЛЮЧЕМ

Retention відповідає на «скільки історії?»: рішення за віком або обсягом, вміст неважливий

Компакція відповідає на «який зараз стан?»: рішення за ключем, вік неважливий

Демо: retention лишив 0 із 12 записів і 0 із 4 ключів



ТРИ ЛАМАЧІ НАЇВНОГО РОЗУМІННЯ

І ДЕФОЛТИ, ЯКІ ЦЕ ПОЯСНЮЮТЬ

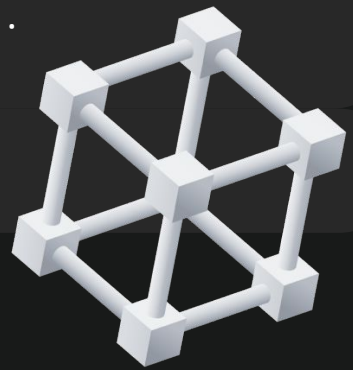
Офсети не перенумеровуються: після компакції в них лишаються діри, і це валідні позиції

Компакція гарантує ЩОНАЙМЕНШЕ останнє значення на ключ, а не «рівно одне»: у демо SKU-1 лишився двічі

Tombstone живе delete.retention.ms (доба): хто відстав більше — маркера не побачить і відновить видалений ключ

Дефолти топіка: cleanup.policy=delete · retention 7 днів · segment.bytes 1 ГіБ · min.cleanable.dirty.ratio 0.5

Активний сегмент НІКОЛИ не компактується — а при 1 ГіБ це назавжди



EXACTLY-ONCE



ЧОТИРИ СЕМАНТИКИ ДОСТАВКИ

Exactly-once на рівні доставки не існує — ні в кого

Що існує насправді

- at-most-once: коміт ДО обробки → втрата при падінні
- at-least-once: коміт ПІСЛЯ обробки → дублікат при падінні
- exactly-once (доставка): не існує на рівні доставки
- exactly-once (ефект): at-least-once + ідемпотентний ефект
- Ціна ефекту: ключ дедуплікації і сховище під нього

Де закінчується EOS Kafka

- EOS = read-process-write у Kafka
- Транзакція: запис + офсети разом
- Зовнішня база в неї не входить
- Далі — ваш ключ дедуплікації
- Ідемпотентність — властивість ОПЕРАЦІЇ; ключ ідемпотентності — ІНСТРУМЕНТ. At-least-once + ідемпотентний ефект = exactly-once, який існує

ТРИ ЦИТАТИ ДОСЛІВНО

ДОКУМЕНТАЦІЯ KAFKA І DEBEZIUM

Kafka docs: “Many systems claim to provide "exactly-once" delivery semantics, but it is important to read the fine print, because sometimes these claims are misleading.” 

Там само: “Exactly-once delivery for other destination systems generally requires cooperation with such systems.” 

Debezium, Exactly-once delivery: “Debezium provides at-least-once delivery guarantees.” 

isolation.level у доках Kafka і в Java-клієнті — read_uncommitted: консюмер за дефолтом бачить і аборчені записи 

Дефолт — властивість вашого клієнта, а не «Kafka»: у librdkafka той самий параметр дорівнює read_committed. Перевіряйте свій

ДЕ САМЕ ЗАКІНЧУЄТЬСЯ ГАРАНТІЯ

ЩО ПОКАЗАВ ЖИВИЙ ПРОГІН

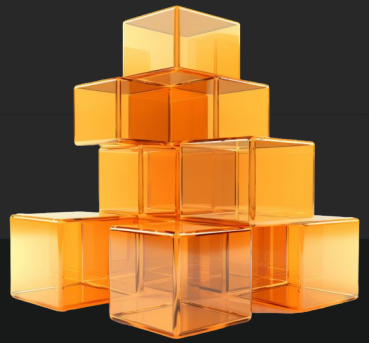
Транзакція Kafka: вхід прочитано, вихід записано, офсет пересунуто — однією операцією

Kill між ефектом і commitTransaction → після рестарту подія приходить удруге

enable.idempotence прибирає лише ретраї продюсера в межах сесії: дві сесії — два записи

At-least-once + ідемпотентний ефект = exactly-once, який існує

Усередині складу пломба працює. Кур ʼєр назовні може приїхати двічі



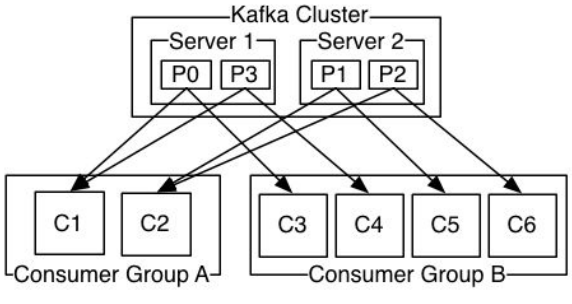
КЛІЄНТИ ДЛЯ NODE

ЩО СТАВИТИ В 2026 РОЦІ

@confluentinc/kafka-javascript 1.10.0 — офіційний клієнт Confluent, KafkaJS-сумісний API

kafkajs 2.2.4 — останній реліз 27.02.2023; @nestjs/microservices тягне саме його як optional peer

У тарболі kafkajs максимальний API key 46 — ні KIP-848, ні share groups



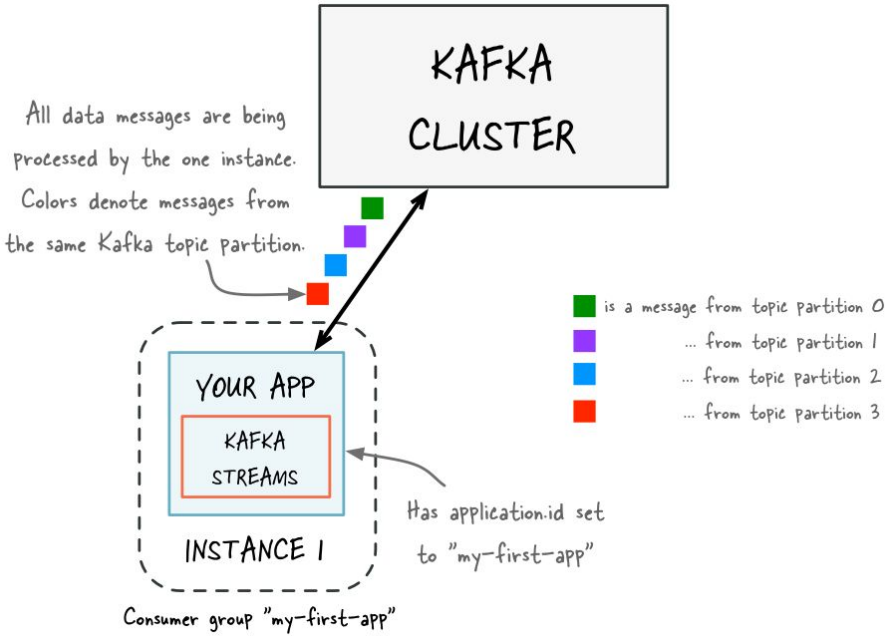
АЛЬТЕРНАТИВИ

ХТО ЕКСПЛУАТУЄ · ВАРТІСТЬ ВИХОДУ · ПАРТИЦІЙНА СТЕЛЯ

- Kinesis: шард 1 МБ/с на запис і 2 МБ/с на читання, retention до 365 днів
- Kinesis: consumer group живе в таблиці DynamoDB, яку адмініструєте й оплачуєте ви
- Pulsar: підписки Shared і Key_Shared дають паралелізм без прив ′язки до кількості партицій
- Redpanda: той самий протокол і ті самі клієнти, але open core і був Jepsen-аудит
- Kafka: екосистема, кадри, портативність. На AWS є ще MSK — керована Kafka

НАСКРІЗНИЙ ПОТІК

OUTBOX → KAFKA → ТРИ НЕЗАЛЕЖНІ ГРУПИ → ІДЕМПОТЕНТНИЙ ЕФЕКТ

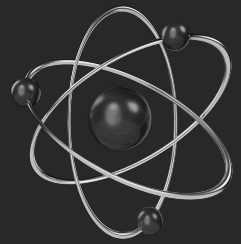


ПІДСУМОК І ТЕСТ

СЬОГОДНІ ДЗ НЕМАЄ — СЬОГОДНІ ТЕСТ НА 2 БАЛИ

Теми тесту

- Топіки, партиції, ключі
- Групи, rebalance, офсети
- Компакція і exactly-once
- Альтернативи: Kinesis, Pulsar



Що забрати із собою

- Event-driven — це обмін фактами про минуле, а не наказами
- Топік із подіями не робить систему event-sourced
- CQRS: читальна модель відстає — це умова угоди, а не баг
- Kafka не прибирає невизначеність — далі ваша ідемпотентність

Q&A

???



YOUR PLACE
TO GROW